

19/DIC/2025



CASO PRACTICO

ELABORACION DE UN DADO ELECTRONICO

Integrantes:

- **Efraín Andrés Vázquez Ramos**
- **Juan Carlos Silva Licea**

Docente:

- **Ing. Daniel Eduardo Cruz Morales**

Materia:

- **Electrónica Básica**

UNIVERSIDAD MULTITECNICA PROFESIONAL

INDICE

1. Introducción

2. Materiales y Herramientas

3. Desarrollo del Proyecto

- 3.1 Fase de Diseño y Simulación (Tinkercad)
- 3.2 Fase de Prototipado Físico (Protoboard)
- 3.3 Fase de Ensamblaje Permanente (Baquelita)

4. Lógica de Programación y carga de software

- 4.1 Proceso de Ejecución en Arduino IDE
- 4.2 Algoritmo y Control de Señal (Debounce)

5. Resultados y Conclusiones

6. Referencias (APA 7)

1. Introducción

El objetivo de este proyecto es la creación de un sistema embebido que emule el comportamiento de un dado convencional de seis caras. Mediante el uso de un microcontrolador Arduino Uno, se genera un número aleatorio activado por un pulsador, el cual se visualiza a través de una matriz de siete LEDs. Este informe documenta el flujo de trabajo desde la validación virtual hasta la soldadura en placa de baquelita.

2. Materiales y Herramientas

Para la ejecución del circuito se utilizaron los siguientes elementos:

- **Microcontrolador:** 1 Arduino Uno R3.
- **Componentes Optoelectrónicos:** 7 LEDs rojos.
- **Resistencias:** 7 de 330 ohms (para LEDs) y 1 de 1k ohms (para pulsador).
- **Control de entrada:** 1 Pulsador de cuatro pines.
- **Soportes:** Protoboard, placa de baquelita perforada y jumpers.
- **Software:** Tinkercad Circuits e IDE de Arduino.

3. Desarrollo del Proyecto

3.1 Fase de Diseño y Simulación (Tinkercad)

Se inició con un esquema virtual para asegurar la integridad de los componentes. Los 7 LEDs se dispusieron en una matriz que permite representar los puntos de un dado (esquinas, centros laterales y un centro único).

- **Configuración de Pines:** Se asignaron los pines digitales del 2 al 8 para los LEDs y el pin 9 para el pulsador.
- **Validación:** Se comprobó que el flujo de corriente no excediera los límites del Arduino mediante el uso de resistencias de 330 ohms, garantizando la vida útil de los LEDs.

3.2 Fase de Prototipado Físico (Protoboard)

Tras el éxito en el simulador, se procedió al montaje en protoboard. En esta etapa se prestó especial atención a:

- **Polaridad:** Identificación correcta de ánodos y cátodos.
- **Conexión del Pulsador:** Se implementó una configuración **Pull-down** utilizando la resistencia de 1k ohms conectada a tierra (GND), asegurando que el pin 9 lea un estado "LOW" estable cuando el botón no está presionado.
- **Bus de Tierra:** Se estandarizó el uso de los rieles laterales para facilitar la conexión común de todos los cátodos y la resistencia del pulsador.

3.3 Fase de Ensamblaje Permanente (Baquelita)

Finalmente, el circuito se trasladó a una placa de baquelita para asegurar su durabilidad.

- **Disposición (Layout):** Se buscó una distribución simétrica de los LEDs para mejorar la estética del dado.
- **Soldadura:** Se aplicó técnica de soldadura de punto para evitar puentes térmicos. Se soldaron primero las resistencias y posteriormente los componentes más sensibles al calor (LEDs).
- **Interconexión:** Se utilizaron cables rígidos para conectar la placa con los pines del Arduino, respetando el orden establecido en el código (Pin 8 para el último LED y Pin 9 para el control del botón).

4. Lógica de Programación y carga de software

El código fuente utiliza la función `random(1, 7)` para generar un valor entre 1 y 6 cada vez que se detecta una señal en el pin de entrada. La estructura se basa en condicionales `if` que activan combinaciones específicas de pines para formar el número visual. Por ejemplo, para el número 1, solo se activa el pin central (LED 4); mientras que para el 6, se activan todos excepto el central.

4.1. Proceso de Ejecución en Arduino IDE Para trasladar la lógica del dado al microcontrolador, se siguió el siguiente procedimiento técnico:

1. **Configuración del Entorno:** Se seleccionó la placa "Arduino Uno" y el puerto COM correspondiente en el menú de herramientas del IDE.

2. **Compilación:** Se realizó una verificación del código para asegurar la ausencia de errores de sintaxis en las estructuras if y en la definición de pines.
3. **Carga (Upload):** El código fue transferido vía USB. Durante este proceso, se validó la comunicación mediante los LEDs *TX/RX* del Arduino, confirmando la escritura exitosa en la memoria Flash del microcontrolador.

4.2. Algoritmo y Control de Señal El programa opera bajo un ciclo infinito (loop) que monitorea constantemente el Pin 9.

- **Gestión de "Debounce" (Antirrebote):** Debido a la naturaleza mecánica del pulsador, se implementó un retraso de 500ms (delay). Esta instrucción es crítica, ya que permite al software ignorar las vibraciones metálicas del botón (rebotes) que ocurren en milisegundos, garantizando que una sola pulsación física genere un único número aleatorio.
- **Generación Aleatoria:** Al detectar la señal en estado ALTO (HIGH), el sistema ejecuta `random(1, 7)`. Este valor determina qué grupo de pines digitales (del 2 al 8) pasará a estado HIGH para iluminar la combinación de LEDs correcta.

5. Resultados y Conclusiones

- **Validación de la Función Aleatoria:** Se realizaron pulsaciones consecutivas registrando los valores en el Monitor Serial. Los resultados demostraron una distribución uniforme de los números del 1 al 6, confirmando que la función `random(1, 7)` opera correctamente y no presenta patrones repetitivos evidentes, cumpliendo con el propósito de un dado electrónico.
- **Prueba de Representación Visual:** Se verificó manualmente que cada número generado correspondiera al patrón de LEDs estándar.
- Para el número 1, se confirmó el encendido único del LED central (Pin 5).
- Para números pares (2, 4, 6), se comprobó la simetría de encendido de los LEDs laterales.
- Para el número 6, se validó que el LED central permaneciera apagado mientras el resto de la matriz se activaba.
- **Respuesta del Pulsador y Debounce:** Se evaluó la sensibilidad del pulsador conectado al Pin 9. Gracias a la resistencia física de 1K Ohms en configuración *Pull-down*, se eliminó el ruido eléctrico, logrando que cada presión física correspondiera exactamente a un nuevo lanzamiento sin falsos disparos.
- **Estabilidad Térmica y Eléctrica:** Tras periodos de uso prolongado (15 minutos de actividad continua), se comprobó mediante inspección táctil y multímetro que las resistencias de 330K Ohms mantuvieron los LEDs en una temperatura de

operación segura y un brillo constante, sin caídas de tensión significativas en el Arduino.

Conclusión.

El proyecto finalizó con un dispositivo funcional y robusto. La transición de Tinkercad a la baquelita fue fluida gracias a la etapa de prototipado en protoboard, que permitió identificar que la resistencia de 1k ohms era fundamental para evitar lecturas erráticas en el pin digital 9. El dado responde con precisión a cada pulsación, demostrando la eficacia del uso de microcontroladores en aplicaciones lúdicas y educativas.

6. Referencias (APA 7)

- **Floyd, T. L. (2007). *Principios de circuitos eléctricos* (8.^a ed.). Pearson Educación.** <https://archive.org/details/floyd-principios-de-circuitos-electricos-8-ed/>
- **Mano, M. M., & Kime, C. R. (2005). *Fundamentos de diseño lógico y de computadoras*. Pearson Educación.** <https://bibliotecavirtualatenead.unad.edu.co/>
- **Platt, C. (2020). *Encyclopedia of Electronic Components: Resistors, Capacitors, Inductors, Switches, Encoders, Relays, Transistors*. Make Community.**

<https://es.scribd.com/document/841619378/Encyclopedia-of-Electronic-Components-Volume-1-Charles-Platt-026-030>

- **Scherz, P., & Monk, S. (2016). *Practical Electronics for Inventors* (4.^a ed.). McGraw-Hill Education.**

<http://instrumentacion.qi.fcen.uba.ar/libro/Scherz.pdf>